

DEPARTMENT OF CSE

COURSE CODE : CSE 402

COURSE TITLE : Operating System Lab

REPORT : 02

REPORT NAME : Implementation of Shortest Job First (SJF) CPU Scheduling Algorithm

SUBMITTED BY:

NAME : SUJANA MAHMUDA NITE

STUDENT ID. : 23101120

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C-1

SUBMITTED TO:

ATIA RAHMAN ORTHI

UAP

DATE OF SUBMISSION: 20-08-2026

Problem Statement

Implement and analyze the FCFS and Non-Preemptive SJF CPU scheduling algorithms for the given set of processes. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, determine the average TAT and WT, and compare the performance of both scheduling algorithms.

Given Data

Process	AT	BT
P1	2	5
P2	3	1
P3	0	2
P4	1	3
P5	1	6

Objective

The main objective of this experiment is to understand the working procedure of the Non-Preemptive Shortest Job First (SJF) scheduling algorithm and compare its performance with the First Come First Serve (FCFS) scheduling method. It also examines how different process execution sequences affect Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and their average values.

Working Principle

SJF (Non-Preemptive): The CPU chooses the process having the smallest Burst Time from all processes that have already arrived. After a process begins, it runs until it finishes. The scheduler then checks the ready processes again and selects the shortest available job.

FCFS: Processes are handled according to arrival order. The earliest arriving process receives the CPU first. When no process is ready, the CPU remains idle until the next process arrives. For both methods, TAT = CT - AT and WT = TAT - BT. Although the formulas remain unchanged, the final values can vary because each algorithm may execute the processes in a different sequence.

Why SJF Can Perform Better

SJF favors shorter ready jobs, allowing several small processes to complete sooner. This commonly lowers the overall average waiting time and turnaround time. For the data used in this lab, SJF gives smaller Average TAT and Average WT than FCFS.

Limitations of SJF

Starvation: A process with a high Burst Time can remain waiting for a long period when shorter jobs continue to become available.

Burst Time Estimation: SJF depends on knowing or predicting CPU burst duration, which may not be exact in practical systems.

Fairness: Since short jobs receive preference, longer jobs may experience less favorable service.

FCFS and Starvation

FCFS generally prevents starvation because processes are served in the same order in which they arrive, so each process eventually receives CPU time. However, FCFS may create a convoy effect when one long process at the front causes several short processes to wait.

Where FCFS and SJF Are Used

FCFS is suitable for straightforward queues such as printer jobs, simple batch systems, ticket or service queues, and cases where arrival-order fairness matters. SJF is more suitable for batch or job scheduling when execution time can be estimated and minimizing average waiting time is a major goal.

Source Code

```
process = ["P1", "P2", "P3", "P4", "P5"]
at = [2, 3, 0, 1, 1]
bt = [5, 1, 2, 3, 6]

n = len(process)

ct = [0] * n
tat = [0] * n
wt = [0] * n
done = [0] * n

time = 0
completed = 0

# SJF - Non Preemptive
while completed < n:
    x = -1

    for i in range(n):
        if at[i] <= time and done[i] == 0:
            if x == -1 or bt[i] < bt[x]:
                x = i

    if x == -1:
        time = time + 1
```

```
    else:
        time = time + bt[x]
        ct[x] = time
        done[x] = 1
        completed = completed + 1

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

print("SJF RESULT")
print(f"{'Process':<10}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")

for i in range(n):
    print(f"{process[i]:<10}{at[i]:<6}{bt[i]:<6}{ct[i]:<6}{tat[i]:<6}{wt[i]:<6}")

sjf_avg_tat = sum(tat) / n
sjf_avg_wt = sum(wt) / n

print()
print("Average TAT =", sjf_avg_tat)
print("Average WT =", sjf_avg_wt)

print()
print()
```

```
# FCFS
processes = [
    ["P1", 2, 5],
    ["P2", 3, 1],
    ["P3", 0, 2],
    ["P4", 1, 3],
    ["P5", 1, 6]
]

processes.sort(key=lambda x: x[1])

time = 0
total_tat = 0
total_wt = 0

print("FCFS RESULT")
print(f"{'Process':<10}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")

for p in processes:
    pid = p[0]
    arrival = p[1]
    burst = p[2]

    if time < arrival:
        time = arrival
```

```
    time = time + burst
    completion = time
    turnaround = completion - arrival
    waiting = turnaround - burst

    total_tat = total_tat + turnaround
    total_wt = total_wt + waiting

print(f"{pid:<10}{arrival:<6}{burst:<6}{completion:<6}{turnaround:<6}{waiting:<6}")

fcfs_avg_tat = total_tat / len(processes)
fcfs_avg_wt = total_wt / len(processes)

print()
print("Average TAT =", fcfs_avg_tat)
print("Average WT =", fcfs_avg_wt)

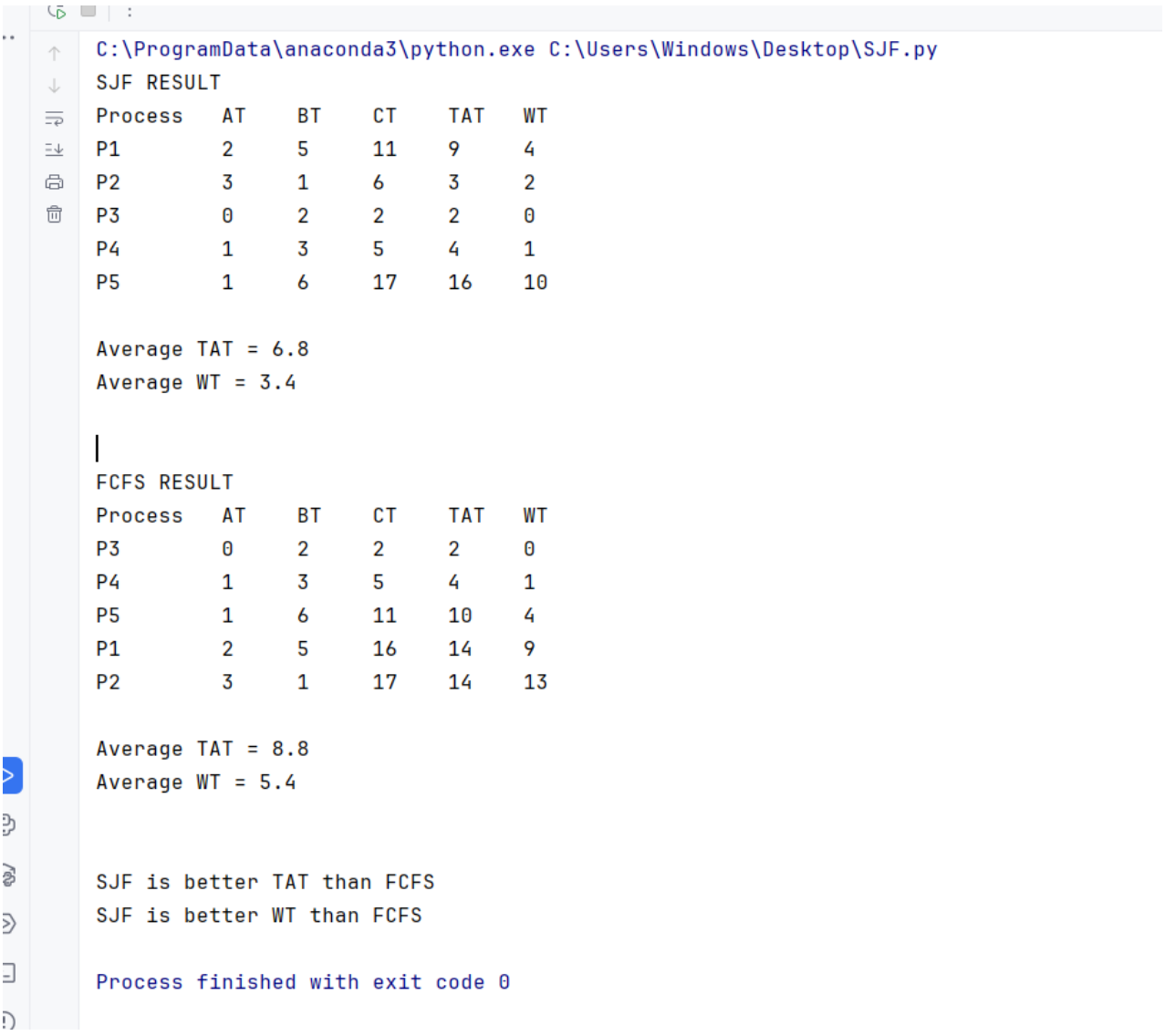
print()
print()

if sjf_avg_tat < fcfs_avg_tat:
    print("SJF is better TAT than FCFS")
else:
    print("FCFS is better TAT than SJF")

if sjf_avg_wt < fcfs_avg_wt:
    print("SJF is better WT than FCFS")
```

```
else:
    print("FCFS is better WT than SJF")
```

Output



```
C:\ProgramData\anaconda3\python.exe C:\Users\Windows\Desktop\SJF.py
SJF RESULT
Process   AT    BT    CT    TAT   WT
P1        2     5     11    9     4
P2        3     1     6     3     2
P3        0     2     2     2     0
P4        1     3     5     4     1
P5        1     6     17    16    10

Average TAT = 6.8
Average WT = 3.4

FCFS RESULT
Process   AT    BT    CT    TAT   WT
P3        0     2     2     2     0
P4        1     3     5     4     1
P5        1     6     11    10    4
P1        2     5     16    14    9
P2        3     1     17    14    13

Average TAT = 8.8
Average WT = 5.4


SJF is better TAT than FCFS
SJF is better WT than FCFS

Process finished with exit code 0
```

Discussion

The FCFS scheduling technique executes processes according to the sequence of their arrival. Based on the given process information, the execution order was P4, P2, P1, P3, and P5. For each process, the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) were calculated. The results showed that the average Turnaround Time was 8.0, whereas the average Waiting Time was 4.6. This experiment helped demonstrate the working principle of FCFS scheduling and showed how the arrival time of each process determines its position in the execution queue and affects its waiting duration.

Conclusion

This experiment successfully demonstrated the implementation of the FCFS CPU scheduling algorithm using Python. The program correctly calculated the execution order along with Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), average TAT, and average WT. From this experiment, a practical understanding of the FCFS scheduling approach was achieved. It was observed that processes are executed strictly in the order of their arrival, meaning that a newly arrived process has to wait until all earlier processes in the queue have finished their execution.

GitHub Link

https://github.com/sujana120/Operating-System-Lab